Ders 2 Genişletilmiş Notu: Sözlük Tabanlı Yöntemler: Statik Sözlük, LZ77, LZ78, LZW ve Görüntü Sıkıştırma

Ders 2 Genişletilmiş Notu: Sözlük Tabanlı Yöntemler: Statik Sözlük, LZ77, LZ78, LZW ve Görüntü Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Olasılık Tabanlı Yöntemlerden Sözlük Tabanlı Yöntemlere
- 2. Statik Sözlük
- 3. Neden Uzun Pattern Kodlamalıyız?
- 4. LZ Ailesi
- 5. LZ77: Kayan Pencere
- 6. LZ78
- 7. LZW
- 8. Sınıf Alıştırması: abbabbaabb
- 9. Görüntü Sıkıştırma ve Dosya Formatları
- 10. JPEG
- Örnek Sorular
- Dersin Toparlaması

1. Olasılık Tabanlı Yöntemlerden Sözlük Tabanlı Yöntemlere

Ders, slaytlarda “Sözlük Tabanlı Yöntemler, Bölüm 6” başlığıyla açıldı (ekrandan, 00:20:00). Hoca önce şimdiye kadar görülenleri hatırlattı: Bu noktaya kadarki yöntemlerin hepsi olasılık tabanlıydı. Verinin istatistiği, yani sembollerin olasılık değerleri çıkarılıyor ve bir model kuruluyordu. Model veriye ne kadar iyi otururs sıkıştırmanın verimi de o kadar artıyordu.

Sözlük tabanlı yöntemlerde ise istatistiksel model kullanılmaz. Hiçbir sembolün ya da karakterin olasılığı hesaba katılmaz; değişken uzunluklu ya da sabit uzunluklu kodlar da kullanılmaz. Bunların yerine sözlükler kullanılır. Hoca burada önemli bir uyarı yaptı: Buradaki sözlük, TDK sözlüğü gibi anlamlı kelimelerin yazıldığı bir sözlük değildir. Sözlükte olabilecek en uzun pattern’lar tutulur. Bir pattern bir, iki, üç, yedi ya da on karakterden oluşabilir; anlamlı da olabilir, anlamsız da. Sözlüğe yazılan bu birimlere token adı verilir.

Sözlükler statik ya da dinamik olabilir:

- **Statik sözlük:** Sözlük hiç değişmez. Yeni pattern eklenemez, var olan silinemez.
- **Dinamik sözlük:** Sözlüğe ayrılan alan dolduğunda daha az kullanılan pattern’lar silinip yerlerine yenileri eklenebilir.

Dinamik sözlükte bir input stream (henüz işlenmemiş veri) ve bir sözlük alanı (işlenip sözlüğe atılmış veri) vardır. Input stream’den gelen veri sözlük alanında aranır. Bu aramanın ayrıntıları yöntemden yönteme değişir.

2. Statik Sözlük

2.1 İngilizce sözlük örneği

En basit statik yöntem, İngilizce dokümanları sıkıştırmak için kurulmuş bir İngilizce sözlüktür. Hoca, bu yöntemin İngilizceye uyduğunu, Türkçeye uymadığını anlattı. İngilizcede kelimelerin aldığı ekler sınırlıdır (-ion, -tion, -s, -ing, -ed gibi). Türkçe ise sondan eklemeli bir dildir; bir kelimeye onlarca ek getirilerek farklı anlamlar kurulabilir. Türkçe için böyle bir sözlük kurmaya kalkılırsa yüz binlerce pattern gerekir.

İngilizce için kurulan sözlükte yaklaşık yarım milyon kelime vardır. Kodlayıcı input stream'den bir kelime okur ve sözlükte arar. Kelime sözlükte varsa sözlükteki sırasını output stream'e yazar; örneğin kelime 121. sıradaysa çıkışa 121 yazılır. İki tarafta da aynı sözlük olduğu için alıcı 121'in hangi kelime olduğunu bilir. Kelime sözlükte yoksa olduğu gibi, ASCII kodlarıyla yazılır.

2.2 İndeks mi, ham kelime mi?

Burada bir problem doğar: Çözücü okuduğu bitlerin bir indeks numarası mı, yoksa olduğu gibi yazılmış bir kelime mi olduğunu nasıl anlayacak? Hoca bunu slayttaki örnekle çözdü (ekrandan, 00:24:00).

Sözlükte $2^{19} = 524\,288$ kelime olsun (yarım milyondan biraz fazla). Bu kadar kelimenin indeksi 19 bitle yazılır. Ayırt etmek için başa bir bayrak (flag) biti konur:

- Kelime sözlükte varsa: bayrak 0, ardından 19 bit indeks. Toplam 20 bit.
- Kelime sözlükte yoksa: bayrak 1, ardından 7 bitte kelimenin karakter sayısı, ardından her karakter için bir byte. Bayrak ve karakter sayısı birlikte bir byte eder. Kelimeler en çok 5 karakter olduğundan toplam $8 + 5 \times 8 = 48$ bit.

Hoca örnek olarak sözlükte olmayan "okul" kelimesini verdi: İlk bit 1 olur, yani "bu kelime sözlükte yok, sonraki 7 biti oku" denir. 7 bitte 4 yazar, yani kelime 4 karakterdir. Sonraki byte'larda o, k, u, l yer alır.

> **Ders sonrası not:** Hoca bir yerde "sıfırsa buradan sonra okuyacağın 20 tane bit sözlükteki indeks sırasını verecek" dedi (00:23:17). Slayta göre bayraktan sonra 19 bit indeks okunur; 20 bit, bayrakla birlikte toplam uzunluktur.

2.3 Kaç kelime sözlükte olmalı?

Bunun üzerine slaytta şu soru soruldu: "5 karakter uzunluğundaki kelimeden kaç tanesi sözlükte bulunmalıdır ki sıkıştırmadan bahsedilebilsin?" (ekrandan, 00:30:00).

Çözüm şöyle kuruldu:

- Bir kelimenin sözlükte bulunma (çakışma) olasılığı P olsun. Sözlükte bulunmama olasılığı $1 - P$ olur.
- Sözlükteki kelime 20 bit, sözlükte olmayan kelime 48 bit tutar.
- Hiç sıkıştırma yapılmasaydı, 5 karakterlik her kelime ASCII ile $5 \times 8 = 40$ bit tutardı.
- N kelime için eşitlik: $N[20P + 48(1 - P)] = 40N$.
- N'ler sadeleşir: $-28P + 48 = 40$, yani $28P = 8$.
- $P = 8/28 \approx 0{,}29$.

Demek ki kelimelerin en az yaklaşık %29'u sözlükte olmalıdır. Tam %29'da ucu ucuna başabaş olunur; %30-35'i bulursa sıkıştırmadan söz edilebilir ($P > 0{,}29$).

Hoca bu yöntemin genelde tercih edilmediğini söyledi. Ancak elinizde belirli bir alanda, sınırlı sayıda kelimeden oluşan dokümanlar varsa kullanılabilir. Slaytta da "özel durumlarda, birkaç yüz kelimeden oluşan statik sözlüklerin kullanılması daha iyi sonuç verecektir" notu vardı. Genel durumda diğer yöntemler daha iyi sonuç verir.

3. Neden Uzun Pattern Kodlamalıyız?

Dinamik yöntemlere geçmeden önce hoca, sözlükte birden fazla karakterden oluşan pattern tutmanın faydasını "string sıkıştırma" örneğiyle gösterdi.

3.1 Tek sembol

Alfabede iki sembol olsun: a_1 ($P_1 = 0{,}8$) ve a_2 ($P_2 = 0{,}2$). Ortalama olasılık $1/2 = 0{,}5$'tir. Hocanın tanımıyla "varyans", her sembolün olasılığının ortalama olasılıktan mutlak farklarının toplamıdır: $|0{,}8-0{,}5|+|0{,}2-0{,}5| = 0{,}6$. Hoca, varyansın önemini önceki derslerden hatırlattı: Varyans büyükse sembollerin kod uzunlukları birbirinden çok farklı olur (biri 2 bit, biri 10 bit gibi) ve dinamik kodlamada daha büyük buffer gerekir; küçükse kod uzunlukları birbirine yakındır.

Tek sembol kodlandığında her sembol 1 bit alır: birine 0, diğerine 1 kodu verilir. Daha azı mümkün değildir, çünkü bit sayısı tam sayıdır.

> **Ek bilgi (derste anlatılmadı):** İstatistikteki varyans, ortalamadan farkların karelerinin ortalamasıdır. Derste ve slaytta "varyans" adıyla kullanılan büyüklük ise mutlak farkların toplamıdır. Buradaki amaç yalnız olasılıkların ne kadar dağınık olduğunu karşılaştırmaktır.

3.2 İki sembollü string'ler

İki sembol yan yana alınırsa dört string yazılabilir. Olasılıklar ve Huffman kodları slayttaki gibidir (ekrandan, 00:35:30):

String	Olasılık	Kod
a_1a_1	$0{,}8 \times 0{,}8 = 0{,}64$	0
a_1a_2	$0{,}8 \times 0{,}2 = 0{,}16$	11
a_2a_1	$0{,}2 \times 0{,}8 = 0{,}16$	100
a_2a_2	$0{,}2 \times 0{,}2 = 0{,}04$	101

- Ortalama olasılık: $1/4 = 0{,}25$.
- Varyans: $|0{,}64-0{,}25|+|0{,}16-0{,}25|+|0{,}16-0{,}25|+|0{,}04-0{,}25| = 0{,}39+0{,}09+0{,}09+0{,}21 = 0{,}78$.
- Ortalama Huffman kod uzunluğu: $1(0{,}64)+2(0{,}16)+3(0{,}16)+3(0{,}04) = 0{,}64+0{,}32+0{,}48+0{,}12 = 1{,}56$ bit/string.
- String iki sembol olduğundan sembol başına $1{,}56/2 = 0{,}78$ bit.

En olası string en kısa kodu, en az olası string'ler en uzun kodu aldı.

> **Ders sonrası not:** Hoca bu sonucu yorumlarken "bir sembolü 0,78 bit ile kodluyorum, bir öncekinde bu 0,6 idi" dedi (00:35:58). 0,6 tek sembol durumundaki varyanstır; tek sembolde ortalama kod uzunluğu 1 bittir. Karşılaştırılması gereken 1 bit ile 0,78 bittir.

3.3 Üç sembollü string'ler

Üçlü string'lerde $2^3 = 8$ durum vardır (ekrandan, 00:40:00):

String	Olasılık	Kod
$a_1a_1a_1$	0,512	0
$a_1a_1a_2$	0,128	100
$a_1a_2a_1$	0,128	101
$a_1a_2a_2$	0,032	11100
$a_2a_1a_1$	0,128	110
$a_2a_1a_2$	0,032	11101
$a_2a_2a_1$	0,032	11110
$a_2a_2a_2$	0,008	11111

- Ortalama olasılık: $1/8 = 0{,}125$.
- Varyans: $|0{,}512-0{,}125|+3|0{,}128-0{,}125|+3|0{,}032-0{,}125|+|0{,}008-0{,}125| = 0{,}387+0{,}009+0{,}279+0{,}117 = 0{,}792$.

- Ortalama Huffman kod uzunluğu: $1(0{,}512) + 3 \cdot 3(0{,}128) + 3 \cdot 5(0{,}032) + 5(0{,}008) = 0{,}512 + 1{,}152 + 0{,}48 + 0{,}04 = 2{,}184$ bit/string.
- Sembol başına: $2{,}184/3 = 0{,}728$ bit.

Slayttaki özet tablo (python ile doğrulandı):

String boyu	Varyans	Sembol başına ortalama kod
1	0,6	1
2	0,78	0,78
3	0,792	0,728

String boyu 4, 5, 6 diye artarsa sembol başına kod uzunluğu daha da küçülür. Örnekteki değerler birbirine yakın çıktı, çünkü alfabede yalnız iki sembol var; gerçek alfabelerde fark daha belirgin olur.

3.4 Sonuç: hece kodlamak

Hoca bunu Huffman üzerinden bağladı: Huffman'da karakter yerine hece kodlansa daha başarılı olunur. Hecelerin çoğu iki ya da üç karakterden oluşur; bazıları tek karakter, bazıları dört karakterdir. Bir karakter yerine bir heceye kod verilirse sembol başına daha az bit harcanır.

Hoca, "sözlük yöntemleri değişken uzunluklu kod kullanmaz dediniz ama burada Huffman kodu yazdınız" itirazını da önceden cevapladı: Bu örnek bir sözlük yöntemi değildir. Yalnızca bir sembol yerine birden fazla sembol kodlanırsa sıkıştırmanın ne kadar başarılı olacağını göstermek için yapıldı. Ana fikir şudur: Mümkün olan en fazla sayıda karakteri tek seferde kodlamak, daha anlamlı bir sıkıştırma performansı verir.

4. LZ Ailesi

Derste LZ77, LZ78 ve LZW anlatıldı. Hoca diğer yöntemlerin bunlardan türediğini söyledi.

Jacob Ziv ve Abraham Lempel 1970'li yıllarda LZ77'yi buldular; ad, soyadlarının baş harflerinden ve 1977 yılından gelir. Bir yıl sonra yöntemi geliştirip LZ78'i ortaya koydular. Bunlar ilk sözlük tabanlı sıkıştırma algoritmalarıdır. Hoca slayttaki türevleri saydı: LZSS (Storer ve Szymanski), LZFG (Fiala ve Greene), LZRW (Ross Williams), LZW (Welch), LZMW (Miller ve Wegman), LZAP (all prefix), LZY ve LZP (prediction).

Bu türevler ayrıntılı anlatılmadı, çünkü çoğu var olan LZ yöntemlerinin belirli parametrelerini değiştirerek ortaya çıkmıştır. Örnek parametreler:

- Buffer boyu: Biri buffer'ın boyunu değiştirip daha iyi sonuç almıştır.
- Arama biçimi: Biri sözlükte doğrusal (linear), biri dairesel (circular) arama yapmıştır.
- Sözlüğün boyu: Biri sabit, biri değişken boyut kullanmıştır.
- Sözlüğün türü: Biri dinamik, biri statik sözlük kullanmıştır.

Hoca en başarılı yöntem olan LZW'yi de anlatacağını söyledi.

5. LZ77: Kayan Pencere

5.1 Ana fikir

LZ77'ye kayan pencere (sliding window) denir. Ana fikir, input stream'den önceden okunmuş bilgiyi sözlük gibi kullanmaktır. Kodlayıcı input için bir pencere oluşturur ve kodlanan değerleri bu pencerenin içinde sağdan sola doğru öteler.

Pencerenin iki parçası vardır:

- **Arama alanı (search buffer):** Solda. Daha önce işlenmiş, kodlanmış verinin yazıldığı yerdir ve sözlük gibi kullanılır. Okunan her yeni sembol burada aranır. Dolana kadar içine veri yazılır; dolunca en eski veri dışarı çıkar, yeni gelen sona girer.
- **Kaynak kod alanı (look-ahead buffer):** Sağda. Input stream buradan akar; henüz kodlanmamış semboller buradadır.

Pratikte arama alanı binlerce byte, kaynak kod alanı onlarca byte uzunluğundadır. Arama alanı her zaman kaynak kod alanından büyüktür.

> **Ders sonrası not:** Hoca pencereyi "bir stack alanı, ilk giden ilk çıkar mantığıyla düşünün" diye tarif etti (00:44:20). İlk giren ilk çıkar (FIFO) davranışı yığına (stack) değil kuyruğa (queue) aittir; pencerenin davranışı bir kuyruğunkidir.

5.2 En uzun eşleşmeyi bulmak

Slayttaki metin "sir sid eastman easily teases sea sick seals" idi. Bir anda arama alanında "sir sid eastman easily t", kaynak kod alanında "eases sea sick seals" bulunur (ekrandan, 00:50:00). Kodlayıcı şöyle çalışır:

1. Kaynak kod alanının ilk sembolü olan "e", arama alanında sağdan sola doğru aranır.
2. İlk "e" "easily" kelimesinde yakalanır. Offset sağdan sayılır; bu "e" 8. konumdadır.
3. Arama sürer; ikinci "e" "eastman" kelimesinde, 16. konumda yakalanır. Başka "e" yoktur.
4. Eşleşme uzatılır: "ea" iki konumda da vardır. "eas" iki konumda da vardır.
5. "ease" aranır; 8. konumdaki "eas"tan sonra "i", 16. konumdakinden sonra "t" gelir. "ease" yoktur.
6. En uzun eşleşme 3 karakterlik "eas"tır. Kodlayıcı en son bulunan çakışmayı referans alarak (16, 3, "e") token'ını hazırlar.

Hoca 8 ve 16'nın ikisinde de üçlü eşleşme olduğunu, en son bulunan eşleşmenin (16) yazıldığını söyledi.

5.3 Token yapısı

LZ77'de output stream'e yazılan token'ın üç alanı vardır:

- **Offset:** En uzun ve en son eşleşmenin arama alanındaki konumu.
- **Length:** Kaç sembolün eşleştiği.
- **Sembol:** Eşleşmeden sonra gelen, eşleşmeyen ilk sembol.

Token yazıldıktan sonra eşleşen semboller ve token'daki sembol arama alanına kaydırılır. Sonraki adım, kaynak kod alanında kalan ilk sembolden başlar.

5.4 Baştan kodlama örneği

Hoca aynı metni baştan kodladı (ekrandan, 01:00:00):

Arama alanı	Kaynak kod alanı	Token
(boş)	`sir␣sid␣eastman␣`	(0, 0, "s")
`s`	`ir␣sid␣eastman␣e`	(0, 0, "i")
`si`	`r␣sid␣eastman␣ea`	(0, 0, "r")
`sir`	`␣sid␣eastman␣eas`	(0, 0, "␣")
`sir␣`	`sid␣eastman␣easi`	(4, 2, "d")

Adım adım:

- "s" arama alanında yoktur (arama alanı boştur). Eşleşme olmadığı için offset 0, uzunluk 0, sembol "s" yazılır ve "s" arama alanına geçer.
- "i", "r" ve boşluk için de aynı şey olur: her biri (0, 0, sembol) olarak yazılır.
- Sonra "s" gelir. Arama alanında bir "s" vardır. "si" aranır; o da vardır. "sid" aranır; arama alanında "si"den sonra "d" gelmez. Eşleşme 2 karakterdir. "s" sağdan 4. konumdadır. Token (4, 2, "d") olur. "s", "i" ve "d" arama alanına geçer.
- Sıradaki boşluk arama alanında vardır; "␣e" yoktur. Hoca bu adımı tahtada (4, 1, "e") olarak yazdı.

Slaytta (0, 0, "s") gibi token'larla tek bir sembol kodlandığı ve bunun iyi bir sıkıştırma sağlamadığı da yazıyordu.

5.5 Alan uzunlukları

Hoca, alanlara ayrılacak bit sayısını sordu ve slayttaki kuralları anlattı (ekrandan, 01:00:00):

- Arama alanının uzunluğu S ise offset alanı $\lceil \log_2 S \rceil$ bit olur. Örneğin $S = 32 = 2^5$ ise offset için 5 bit ayrılır.
- Pratikte arama alanı birkaç bin byte olduğundan offset 10-12 bit olur. LZ türevlerinden bazıları 10, bazıları 12 ya da 15 bitle daha iyi sonuç aldığını söyler; bu, türevlerin oynadığı parametrelerden biridir.
- Kaynak kod alanının uzunluğu L ise length alanı $\lceil \log_2(L-1) \rceil$ bit olur. Pratikte birkaç on byte'lık bir kaynak kod alanı için birkaç bit yeter.
- Sembol alanı 8 bittir, çünkü buraya her zaman tek karakter yazılır.
- Yaklaşık bir token uzunluğu $11 + 5 + 8 = 24$ bittir.

S ve L değerleri, elimizdeki girdinin içeriğine göre denemelerle de belirlenebilir.

5.6 Kod çözme

Alıcı token'ları sırayla okur. (0, 0, "s") ile çakışma yoktur, "s" yazılır; aynı şekilde "i", "r" ve boşluk yazılır. (4, 2, "d") gelince 4 konum geri gidilir, orada "s" vardır; 2 sembol ("si") kopyalanır ve arkasına "d" eklenir. Böylece kodlayıcının yaptığı iş tersinden yapılarak metin açılır.

Hoca kod çözücünün kodlayıcıdan daha basit olduğunu vurguladı. Kod çözücü pencere boyutunda bir buffer tutar, token'ı okur, çakışmanın olduğu yere gider, uzunluk kadar karakteri alır, token'ın üçüncü alanını ekler ve hepsini output stream'e yazar.

5.7 Yöntem ne zaman başarılı olur?

Yakalanan eşleşmeler ne kadar uzunsa yöntem o kadar başarılıdır. Hoca örnek verdi: Arama alanında daha önce geçmiş 16 karakterlik bir ifade yeniden gelirse tamamı tek token ile kodlanır. Offset için 10, uzunluk için 12 bit ayrılsa bile yaklaşık 30 bitlik bir token, $16 \times 8 = 128$ bitlik ifadenin yerini alır.

Arama alanı büyütülürse daha uzun eşleşmeler bulunabilir, ama token'daki offset değerleri de büyür. Türevlerin bir kısmı offset ve length alanlarını değişken boylu yapar.

5.8 Doğrusal ve dairesel pencere

Hoca altı gözlü bir pencereye A, B, C, D, E, F, G harflerinin girdiğini düşündürdü. Doğrusal pencerede her yeni harf girince bütün harfler bir göz kayar. Pencere dolunca G gelince ilk giren A dışarı atılır; içeride B, C, D, E, F, G kalır.

Dairesel kuyrukta (circular queue) ise iki pointer vardır: start (s) ve end (e) (ekrandan, 01:10:00). Karakterler end pointer'dan eklenir, start pointer'dan çıkarılır. Pencere dolduğunda yeni karakter, kaydırma yapılmadan en eski karakterin yerine yazılır; end pointer oraya gelir, start pointer bir ilerler. Slayttaki örnekte "sid␣eastman␣easi" penceresine "l" gelince "s"nin yerine yazılır ve pencere "lid␣eastman␣easi" olur; sonra "y" ve diğerleri aynı biçimde eski karakterlerin yerine yazılır.

Doğrusal ya da dairesel olması da parametrik bir seçimdir. Hoca'ya göre başarı performansı değişmez, ama dairesel yapı kaydırma yapmadığı için hız kazandırır. Pencere çok uzunsa dairesel yapı daha mantıklıdır; birkaç bitlik bir alan ayrılmışsa doğrusal yapı yeter.

6. LZ78

6.1 LZ77'den farkı

LZ78'de Lempel ve Ziv büyük değişiklikler yaptı (ekrandan, 01:30:00):

- LZ77'deki gibi kayan pencere, arama alanı ve kaynak kod alanı kullanılmaz.
- Önceden karşılaşılan string'lerden oluşan bir sözlük kullanılır.
- Sözlük boş başlar. Boyu kullanılabilir hafızayla sınırlıdır ya da sözlüğe eklenebilecek pattern sayısıyla sınırlandırılabilir. Pattern'lar yine anlamlı olmak zorunda değildir; hece ya da parça olabilir.

- Kodlayıcı output stream'e iki alanlı bir token yazar: ilk alan sözlüğün sıra belirleyicisi (kaçıncı sırada olduğu), ikinci alan semboldür.

Algoritma şöyledir:

1. Input stream'den gelen w sembolü sözlükte aranır.
2. Çakışma yoksa w sözlüğün son sırasına yerleşir ve sıra belirleyicisi 0 olur.
3. Çakışma varsa bir sonraki sembol k, w'ye eklenerek wk oluşturulur ve sözlükte arama sürer.
4. Bu işlem en uzun string sözlükte bulunana kadar sürer.
5. wk sözlükte bulunmazsa sözlüğün son sırasına yerleştirilir.
6. Token'ın ilk alanına en son çakışmanın olduğu sıra, ikinci alanına k yazılır.

6.2 Örnek: "sir sid eastman easily teases sea sick seals"

Sözlük 0. sırada null ile başlar (ekrandan, 01:40:00). Hoca renk renk işaretleyerek adım adım ilerledi:

Sıra	Sözlük	Token	Açıklama
0	null		Başlangıç
1	s	(0, s)	s yok
2	i	(0, i)	i yok
3	r	(0, r)	r yok
4	␣	(0, ␣)	boşluk yok
5	si	(1, i)	s 1. sırada var, si yok
6	d	(0, d)	d yok
7	␣e	(4, e)	boşluk 4. sırada var, ␣e yok
8	a	(0, a)	a yok
9	st	(1, t)	s var, st yok
10	m	(0, m)	m yok
11	an	(8, n)	a 8. sırada var, an yok
12	␣ea	(7, a)	␣e 7. sırada var, ␣ea yok
13	sil	(5, l)	si 5. sırada var, sil yok
14	y	(0, y)	y yok
15	␣t	(4, t)	boşluk var, ␣t yok

15. satırı hoca öğrencilere sordu ve birlikte bulundu: Sırada boşluk vardır; boşluk sözlükte 4. sıradadır; "␣t" yoktur; bu yüzden 15. sıraya "␣t" yazılır ve token (4, t) olur. Bu tablo python ile yeniden üretildi.

İşlem metnin sonuna kadar böyle sürer.

6.3 Sözlüğün boyu

Sözlük dipsiz bir kuyu değildir. Örneğin indeks en çok 10 bit olacaksa sözlükte 2^{10} pattern olabilir; o sayıya gelinince ekleme durur. İndeks 4 bit ise 0'dan 15'e kadar 16 değer yazılabilir ve sözlük orada dolar. Sınır, eldeki hafızaya göre de konabilir.

6.4 Trie ağacı

Hoca, sözlükte sıralı arama yapmanın yavaş olacağını belirtti. Örneğin "sil" aranırken önce "s", sonra "si", sonra "sil" aranır; her seferinde sözlüğü baştan taramak gerekir. Bunu hızlandırmak için trie ağaçları kullanılır; slayta göre sözlük için en iyi veri yapısı budur (ekrandan, 01:42:30).

Trie'nin kökü null'dır. Kökten alfabe boyu kadar dal çıkar: İngilizcede 26, Türkçede 29 harf (boşlukla birlikte bir fazla). Her düğümün altında, o pattern'ı izleyen karakterler bulunur. Slayttaki ağaçta kökün altında boşluk (4), a (8), c, d, e, i, k, m, r, s, y; boşluğun altında s, e (7), t (15); "␣e"nin altında a (12); "s"nin altında i (5), t (9); "si"nin altında l (13) gibi düğümler vardır.

“␣se” gibi üçlü bir pattern aranırken sözlük sırayla taranmaz: Önce kökten boşluğun düğümüne gidilir, onun altından “s”ye, onun altından “e”ye inilir. Her karakterden sonra her karakter gelmediği için ağaç tam dolu olmaz; hoca Türkçede “ğ”den sonra “g” gelmemesini örnek verdi. Hafıza büyükse ağaca daha uzun (üçlü, dörtlü, beşli) pattern’lar yazılabilir; pattern ne kadar uzunsa ağacın derinliği o kadar artar.

6.5 Sözlük dolduğunda

Sözlüğe ekleme sözlük dolana kadar sürer. Sözlük dolunca türevlere göre üç yol vardır (ekrandan, 01:42:30):

- **Dondurma (FROZEN):** Sözlüğe artık yeni bir şey eklenmez. Sözlükte olanlar token olarak, olmayanlar ASCII karşılığıyla gönderilir.
- **Silme:** Hafızada yer kalmadıysa bütün ağaç silinir ve boş bir ağaçla yeniden başlanır.
- **En az kullanılanı çıkarma:** Ağaçta en seyrek geçen pattern’lar sözlükten çıkarılır, yerlerine yenileri konur.

Hoca üçüncü yolu bir tarih kitabıyla açıkladı. İlk çağı anlatan bölümlerde kullanılan kelimelerle günümüzü anlatan bölümlerde kullanılan kelimeler farklıdır. Eski tarihte “cumhuriyet” kelimesi hiç geçmez; “cu”, “cum” gibi pattern’lar hiç oluşmamıştır. Diyelim ki 12. bölümde yakın tarih başlar. O zaman artık kullanılmayan eski pattern’lar silinip yerine yeni pattern’lar konur. Bu zahmetli bir iştir, ama bunu yapan türevler vardır.

Hoca özet olarak şunu söyledi: Statik sözlük sondan eklemeli dillere pek uygun değildir. LZ77’de token üç alanlıdır ve sözlük, doğrusal ya da dairesel ilerleyen kayan bir penceredir. LZ78 token alanlarını ikiye indirir, hafızada daha az yer kaplar ve sözlükte daha fazla şey tutar.

7. LZW

7.1 Genel yapı

LZW, LZ’ye Welch’in eklenmesiyle oluşur; Welch soyadıdır. Hoca, LZ’ler içinde en performanslı yöntemin bu olduğunu ve bilinen birçok sıkıştırma formatının içinde kullanıldığını söyledi.

LZW’nin özellikleri:

- Sözlüğe burada codebook denir. Hoca, codebook’un “afilli bir isim ama bildiğin dictionary” olduğunu söyledi. Codebook kullanılıyorsa içindekilere code word, dictionary kullanılıyorsa pattern denir.
- Codebook dinamik olarak, metin okundukça oluşturulur.
- Kodların boyutu codebook’un uzunluğuna bağlıdır. Codebook boyu 8 ise içine 8 code word yerleşir.
- Codebook’un başlangıcını alfabe oluşturur. Alfabede kaç farklı sembol varsa önce onlar yazılır; sonra ikili, üçlü, dörtlü pattern’lar eklenir.
- Çıkışa yalnız codebook indeksi yazılır.

7.2 Örnek: abbbaacbbabbaacacaabb

Slayttaki örnekte input stream “abbbaacbbabbaacacaabb”, alfabe {a, b, c} idi (ekrandan, 01:50:00). Codebook 0 = a, 1 = b, 2 = c ile başlar; slayttaki tabloda her indeksin 4 bitlik ikili karşılığı da (0000, 0001, …) yazılıydı.

Kodlama adımları (sınırsız codebook):

1. “a” codebook’ta var; “ab” yok. “a”nın kodu 0 çıkışa yazılır, 3. sıraya “ab” eklenir. “b”den devam edilir.
2. “b” var; “bb” yok. Çıkış 1, 4. sıraya “bb”.
3. “b” var, “bb” var; “bba” yok. Çıkış 4, 5. sıraya “bba”.
4. “a” var; “aa” yok. Çıkış 0, 6. sıraya “aa”.
5. “a” var; “ac” yok. Çıkış 0, 7. sıraya “ac”.
6. “c” var; “cb” yok. Çıkış 2, 8. sıraya “cb”.
7. “bba” var; “bbab” yok. Çıkış 5, 9. sıraya “bbab”.

8. “bba” var; “bbaa” yok. Çıkış 5, 10. sıraya “bbaa”.
9. “ac” var; “aca” yok. Çıkış 7, 11. sıraya “aca”.
10. “aca” var; “acaa” yok. Çıkış 11, 12. sıraya “acaa”.
11. Kalan “abb”: “ab” var; “abb” yok. Çıkış 3, 13. sıraya “abb”.
12. Kalan “b”: çıkış 1.

Çıkış: 0 1 4 0 0 2 5 5 7 11 3 1. Hocanın tahtadaki tablosu 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa ile aynıdır (ekrandan, 02:10:00). Bütün adımlar python ile doğrulandı.

> **Ders sonrası not:** Hoca son adımda “ABB sözlükte var, 5” dedi (01:56:40). O anda “abb” codebook’ta yoktur ve 5 numara “bba”dır. Doğrusu, kalan “abb”nin “ab” (3) ve “b” (1) olarak kodlanmasıdır.

7.3 Kod çözme

Çözücü de yalnız alfabeyle (0 = a, 1 = b, 2 = c) başlar ve gelen indeksleri okuyarak aynı codebook’u kurar. Çıkıştaki ilk değer 0’dır, yani “a” yazılır. Bir sonraki değer 1 (“b”) gelince, önceki string ile bunun ilk sembolü birleştirilip “ab” 3. sıraya eklenir. Sonra 4 gelir; o anda 4 henüz kurulmamıştır, ama hoca bunun önceki “b”nin arkasından bir “b” daha gelmesi demek olduğunu, yani “bb” olduğunu açıkladı. Çözücü bu şekilde, sözlüğe eklenmesi gerekenleri ekleyerek kodlayıcıdakinin aynısını kurar. Hoca bu kısmı anlatırken biraz karıştırmış olabileceğini kendisi söyledi.

> **Ek bilgi (derste anlatılmadı):** Çözücünün henüz kurmadığı bir indeksi alması, kodlayıcının bir code word’ü eklediği adımda hemen kullandığı durumda olur. Bu durumda yeni code word, önceki string ile önceki string’in ilk sembolünün birleşimidir (burada “b” + “b” = “bb”).

7.4 Sınırlı codebook

Sonra hoca codebook’u 8 ile sınırladı: Yalnız 0-7 arası sekiz code word yazılabilir. Kodlama 7. sıraya “ac” eklenene kadar aynıdır. Sonra codebook dolar ve yeni bir şey eklenmez; yalnız codebook’ta var olanlar kullanılır. Python ile doğrulanan çıkış: 0 1 4 0 0 2 5 5 7 7 6 4.

> **Ders sonrası not:** Hoca sınırlı codebook’ta son adımları “AB, 3” ve “B, 1” olarak yazdı (02:02:13). Python ile yapılan doğrulamaya göre iki “ac”den (7, 7) sonra kalan “aabb”dir; bu “aa” (6) ve “bb” (4) olarak kodlanır.

Hoca sözlük boyunun etkisini doğal dil metniyle açıkladı. Codebook’a sınır konmazsa codebook belki dokümanın sonuna kadar dolmaz. Doğal dilde belirli pattern’lar sürekli tekrar ettiği için dokümanın 10. ya da 20. sayfasına gelindiğinde sözlüğe pek yeni şey eklenmez; ancak yeni kelimeler girdikçe eklenir.

8. Sınıf Alıştırması: abbabbaabb

Hoca boş bir slayt açtı ve öğrencilerden kağıt kalemle denemelerini istedi. Input “abbabbaabb”, alfabe $\Sigma = \{a, b\}$, codebook boyu 5 idi (ekrandan, 02:08:45).

8.1 LZW, codebook 5

Codebook 0 = a, 1 = b ile başlar; 2, 3 ve 4 boştur.

1. “a” var; “ab” yok. Çıkış 0, 2 = “ab”.
2. “b” var; “bb” yok. Çıkış 1, 3 = “bb”.
3. “b” var; “ba” yok. Çıkış 1, 4 = “ba”. Codebook doldu.
4. “a” var, “ab” var; “abb” yok ama eklenemez. Çıkış 2.
5. “b” var, “ba” var; “baa” yok. Çıkış 4.
6. “a” var, “ab” var; “abb” yok. Çıkış 2.
7. Kalan “b”: çıkış 1.

Çıkış: 0 1 1 2 4 2 1. Hocanın tahtadaki sonucuyla aynıdır ve python ile doğrulandı.

Codebook sınırlı olmasaydı 5 = “abb”, 6 = “baa” eklenir ve çıkış 0 1 1 2 4 5 olurdu; hoca tahtaya 5 = abb ve “baa” satırlarını ekledi (ekrandan, 02:11:00). Bu da python ile doğrulandı.

8.2 LZ78

Sonra hoca aynı girdiyi LZ78 ile kodlamaya geçti. Sözlük 0 = null ile başlar. Tahtada (0, a), (0, b), (2, a), (1, b) token'ları ve 1 a, 2 b, 3 ba, 4 ab, 5 ba(a) satırları yazıldı (ekrandan, 02:11:00). Hoca bu sırada hangi sembolde olduğunu karıştırdığını kendisi söyledi.

> **Ders sonrası not:** Python ile yapılan doğrulamaya göre "abbabbaabb"nin LZ78 kodlaması şöyledir: 1 = a (0, a); 2 = b (0, b); 3 = ba (2, a); 4 = bb (2, b); 5 = aa (1, a); kalan "bb" sözlükte 4. sırada olduğu için son token (4, -). Tahtadaki dördüncü token (1, b) ve "ab" satırı, girdide 5. karakter "b" olduğu hâlde "a" okunmasından kaynaklanıyor.

8.3 LZ77 ile kısa karşılaştırma

Hoca aynı girdinin LZ77 ile nasıl başlayacağını da sözlü gösterdi: "a" ve "b" için çakışma yoktur. Sonra "b" arama alanında 1. konumda eşleşir, bir sembol eşleşir, ardından "a" gelir; token (1, 1, a) olur. Sonrasında ikili, üçlü, dörtlü eşleşmeler aranarak devam edilir. Bu kısım derste tamamlanmadı (çözümü Örnek Sorular bölümünde).

9. Görüntü Sıkıştırma ve Dosya Formatları

9.1 Ön işlem

Son bölümde hoca görüntü sıkıştırmadan söz etti. Görüntüde önce bir ön işlem adımı vardır; ardından görülen bütün yöntemler görüntüye uygulanabilir. Ama her yöntem metinde başarılı olmadığı gibi, görüntüde de her yöntem başarılı olmaz; en iyisi aranır.

Ön işlem çoğunlukla renk sayısını azaltmaktır. Görüntü işleme, veri madenciliği ya da doğal dil işleme derslerinde görülen kümeleme yöntemleri bu işte kullanılabilir. True color bir resimde Red, Green, Blue için 8'er bit, toplam 24 bit vardır, yani 2^{24} renk. Göz bu kadar rengi ayırt edemez; ayırt edemeyeceği renkler resimden çıkarılabilir.

9.2 Formatlar

- **GIF:** Hocaya göre 1987'de CompuServe tarafından geliştirildi ve özünde LZW vardır. 256 farklı renkle çalıştığı için renkli fotoğraflarda yetersiz kalır; gri tonlu resimlerde daha başarılıdır. LZW patentli bir yöntem olduğu için ticari uygulamada telif ödenmesi gerekiyordu; bu yüzden ticari amaçla pek kullanılmadı.
- **TIFF:** Hoca Adobe tarafından geliştirildiğini söyledi. 5.0 sürümünde LZW kullanılır; 1992'deki 6.0 sürümüne JPEG sıkıştırması eklenmiştir. LZW ile kayıpsız, JPEG ile kayıplı sıkıştırma yapılır. Çok renkli dosyalarda iyi performans vermese de hızı yüzünden tercih edilir.
- **PNG (Portable Network Graphics):** GIF ve TIFF'e karşı geliştirilmiş açık bir formattır, ticari amaçla da kullanılabilir. 48 bit gerçek renge ve gri tonda 16 bite kadar destek verir. Altında zlib ve deflate vardır.
- **ZIP ve GZIP:** Birden fazla dosyayı (metin ya da görüntü) sıkıştırabilir. Veri bloklara ayrılır; blok büyüklüğüne yazılımı kodlayan karar verir, bloklar sabit ya da değişken uzunluklu olabilir. İçinde LZ77'nin bir türevi ile statik Huffman kullanılır. Türevdir, çünkü 32 KB'lık bir kayan sözlük ve 258 byte'lık bir kaynak kod alanı kullanır; bu boyutlar değiştirilebilir (örneğin 64 KB).
- **PKZip, PKArc ve benzerleri:** Hoca birinde LZW ile Huffman'ın, birinde Shannon-Fano ile LZ77'nin birlikte kullanıldığını söyledi.

  > **Ders sonrası not:** TIFF büyük olasılıkla ilk olarak Aldus tarafından geliştirildi; Adobe, Aldus'u satın aldıktan sonra formatın sahibi oldu.

Hoca'ya göre bu formatların hepsi görülen yöntemlerin birleşimidir. Biri RLE ile ön işlem yapıp ardından Huffman uygular; biri önce quantization ile renkleri azaltıp ardından görülen yöntemlerden birini kullanır. Tekrar eden pattern'lı resimlerde, örneğin siyah zemin üzerinde beyaz puantiyeli bir resimde, uzun siyah piksel dizileri ve kısa beyaz diziler vardır; burada LZW de RLE de başarılı olabilir. Veriye uygun yöntemler denenir ve en iyi sonucu veren seçilir.

10. JPEG

10.1 JPEG ve JPEG 2000

JPEG'in farklı versiyonlarında kayıp oranı ayarlanabilir: "%25, %40 ya da %50 kaybı kabul ediyorum" denebilir. Atılan bilgi, gözün algılamakta zorlandığı renklerdir. Örneğin göz bazı yeşil tonlarını ayırt edemez; bunlar gözün ayırt edebildiği, daha sık kullanılan yeşillerle ifade edilir. Renk sayısı azalınca prefix kodlama daha az kod üretir, başlık (header) da küçülür.

Hoca'ya göre JPEG içinde Huffman kullanılır ve sıkıştırma kayıplı ya da kayıpsız olabilir. DCT (Discrete Cosine Transform) float sayılarla ve bölme işlemleriyle çalıştığı için kayıplar olabilir. JPEG 2000 ise wavelet dönüşümünü kullanır; hoca wavelet dönüşümünün DCT'den daha başarılı olduğunu ve JPEG 2000'in kayıpsız sıkıştırma yaptığını söyledi.

> **Ders sonrası not:** Hoca DCT'yi anlatırken "ben düşük frekansları resimden çıkarmaya çalışıyorum" dedi (02:34:17). Slayta ve hocanın sonraki anlatımına göre çıkarılan, gözün algılayamadığı yüksek frekanslardır.

> **Ders sonrası not:** Hoca JPEG 2000'in "tamamıyla kayıpsız" olduğunu söyledi. JPEG 2000 büyük olasılıkla hem kayıplı hem kayıpsız sıkıştırma kipini destekler.

JPEG, DCT ve quality factor'e göre önceden hazırlanmış Q (quantization) matrislerini kullanır. 24 bit renk derinliğindeki resimlerde çok iyi sonuç verir; bu yüzden dijital fotoğrafçılıkta kullanılır. JPEG birçok görüntü sıkıştırma yönteminin bir araya getirilmesiyle oluşmuş bir standarttır. Arka arkaya birkaç yöntem kullandığı için sıkıştırma ve açma süresi tek algoritmalı yöntemlere göre daha uzun olabilir; ama hocaya göre fark mikrosaniyeler düzeyindedir ve günlük kullanımda önemsizdir. Yerden mi, zamandan mı kazanılacağına kullanıcı karar verir.

10.2 JPEG mimarisi

Slayttaki mimari, bir deniz feneri resmi üzerinden anlatıldı (ekrandan, 02:40:20):

1. **RGB → YUV (isteğe bağlı):** Renk bilgisi RGB ise YUV'a çevrilir. Sıkıştırma RGB üzerinde de yapılabilir. RGB'yi YUV'a ya da HSV'ye çeviren matrisler literatürde hazırdır. Y bileşeni: $Y = 0{,}299R + 0{,}587G + 0{,}114B$.
2. **Örneklemeyi azaltma (down-sampling).**
3. **8×8 bloklara bölme:** Her renk kanalı (Y, U, V) ayrı ayrı 8×8 bloklara bölünür. Hoca bunu, RLE ile sıkıştırma yaparken resmi küçük bloklara bölme önerisine benzetti.
4. **DCT:** Her blok ayrı ayrı DCT'den geçirilir; piksel bilgisi zaman etki alanından frekans etki alanına çevrilir.
5. **Nicemleme (quantization):** DCT sonuçları seçilen kayıp oranına ait Q matrisiyle nicemlenir.
6. **Zigzag tarama ve kodlama:** Katsayılar köşegen (zigzag) düzende taranır. DC katsayılarına DPCM, AC katsayılarına RLE uygulanır; slayta göre ardından QM kodlaması gelir. Hoca sözlü anlatımda burada (dinamik) Huffman kullanıldığını söyledi.
7. **JPEG dosyası:** Başlık, tablolar (nicemleme ve kodlama tabloları) ve veri.

Açarken her işlemin tersi yapılır: önce nicemlemenin tersi, sonra DCT'nin tersi, sonra renk uzayı dönüşümünün tersi.

10.3 Renk dönüşümü ve örneklemeyi azaltma

RGB'den YUV'a geçişte parlaklık (luminance) ve renk (chrominance) bileşenleri ayrılır. İnsan gözü parlaklığa daha duyarlı, renge daha az duyarlıdır. Bu yüzden U ve V değerlerini bir miktar azaltmak gözde fark edilir bir bozulmaya yol açmaz. Hoca, resmin 2×2'lik (4 piksellik) parçalara bölündüğünü ve U, V değerlerinin bu parçalarda değiştirildiğini anlattı.

> **Ek bilgi (derste anlatılmadı):** Yaygın bir down-sampling biçiminde (4:2:0), her 2×2 piksellik parça için Y değerleri dört piksel için ayrı ayrı tutulur, U ve V ise parça başına birer değerle temsil edilir. Böylece renk bilgisi dörtte birine iner.

10.4 DCT

Amaç, insan gözünün göremediği yüksek frekansları resimden elemektir (ekrandan, 02:46:40):

- Resim 8×8 bloklara ayrılır; blok içindeki renk değerleri 0-255 arasındadır.
- DCT $[-128, 127]$ aralığında çalıştığından bütün renk değerlerinden 128 çıkarılır. Elde edilen matrise T matrisi denir.
- Her bloğa ayrı ayrı DCT uygulanır. Bunun için $N \times N$ boyutlu bir M matrisi hesaplanır:

$$M_{i,j} = \begin{cases} \frac{1}{\sqrt{N}} & i = 0 \\ \sqrt{\frac{2}{N}} \cos\left[\frac{(2j+1)i\pi}{2N}\right] & i > 0 \end{cases}$$

- DCT katsayılarından oluşan D matrisi M ve T'nin çarpımıyla bulunur; slaytta $D = M^T T M$ yazıyordu.
- Sonuçta 64 katsayı çıkar. İlki (D(0,0)) DC, diğerleri AC katsayısıdır. Düşük frekanslı değerler matrisin sol üst köşesinde, yüksek frekanslı değerler sağ alt köşededir.

  Ders sonrası not: Hoca sözlü anlatımda sıfırıncı satır sıfırıncı sütundaki değere "AC", diğerlerine "DC" dedi ve D matrisinin sol üst köşesinin yüksek frekanslı, sağ alt köşesinin alçak frekanslı olduğunu söyledi (02:44:29, 02:47:19). Slayta göre ilk katsayı DC'dir, diğerleri AC'dir; düşük frekanslar sol üstte, yüksek frekanslar sağ alttadır.

  Ders sonrası not: Slayttaki M tanımında i satırı frekansı gösterir. Bu tanımla python'da denendiğinde doğru iki boyutlu DCT'yi $D = MTM^T$ verdi; slayttaki $D = M^T T M$ sırası bu tanımla doğru sonuç vermedi.

10.5 Nicemleme ve geri dönüş

D matrisine kayıp oranına göre seçilen Q matrisi uygulanır: %25 kayıp için bir matris, %50 kayıp için başka bir matris. D'deki her değer Q'daki karşılığına bölünüp yuvarlanır ve C matrisi elde edilir. Bölmede ondalıklı sayılar çıktığı ve yuvarlandığı için bilgi kaybolur. Hoca örnek verdi: 2,5 sayısı 2'ye yuvarlanırsa artık orijinal 2,5'e dönülemez. Q matrisleri hazırdır, internette bulunabilir.

Nicemleme sonrası sağ alt köşedeki, gözün algılayamadığı değerler sıfırlanmaya çalışılır. Ardından katsayılar zigzag taramayla okunur. Resim kaç parçaya bölündüyse o kadar C matrisi vardır; bunlar yan yana konarak sıkıştırılmış resim oluşur.

Geri dönerken C matrisi aynı Q matrisiyle çarpılır, M matrisleriyle ters DCT yapılır ve 128 eklenir. Yuvarlama kayıpları yüzünden orijinal matris tam elde edilmez, ama göz bu küçük farkları algılamaz. Örneğin bir bitmap JPEG'e çevrilip yeniden bitmap'e çevrilirse ufak kayıplar olur. Nicemleme adımı (bölme ve yuvarlama) yapılmazsa kayıp olmaz ve orijinal veriye dönülebilir.

10.6 Özet ve kalite ölçüsü

Hoca JPEG'i şöyle toparladı: Gerekirse palet değiştirilir (göz parlaklığa daha duyarlı olduğu için parlaklık ön plana çıkarılır), down-sampling yapılır, değerler 128 kaydırılır, DCT uygulanır, yüksek frekanslar elenir, kayıp isteniyorsa Q matrisleri uygulanır, ardından RLE ve Huffman uygulanır ve sıkıştırılmış JPEG dosyası oluşur.

Hoca bütün bunlardan şu sonucu çıkardı: Görüntünün kendine özel bir sıkıştırma yöntemi yoktur; yalnız görüntüye özel ön işlemler yapılabilir. Örneğin k-means kümelemeyle renkler 16 kümeye ayrılabilir. Her kümede bir renk ve tonları (mavi ve tonları, kırmızı ve tonları) bulunur; kümenin ortalaması alınarak bütün kırmızı tonları tek bir kırmızıyla ifade edilir. Daha hassas sonuç isteniyorsa kümeler alt kümelere bölünebilir. Bundan sonrasında metinde kullanılan yöntemler kullanılır; redundancy yeterince azalmamışsa ardından bir sıkıştırma yöntemi daha uygulanır.

Görüntüde sıkıştırma başarısı, önceki derslerde anlatılan sinyal-gürültü oranı (SNR) ve PSNR gibi ölçülerle değerlendirilir. Bunlar orijinal ve sıkıştırılmış veri arasındaki farkların karesine dayanır; en küçük kareler yönteminde olduğu gibi hata en aza indirilmeye çalışılır.

Ek bilgi (derste anlatılmadı): PSNR genelde PSNR $= 10\log_{10}(\text{MAX}^2/\text{MSE})$ ile hesaplanır. 8 bitlik görüntüde MAX = 255'tir; MSE, orijinal ve açılmış görüntü arasındaki piksel farklarının karelerinin ortalamasıdır. PSNR büyüdükçe açılan görüntü orijinale daha yakındır.

Örnek Sorular

Soru 1 (ekrandan, 00:30:00): 5 karakter uzunluğundaki kelimeden kaç tanesi sözlükte bulunmalıdır ki sıkıştırmadan bahsedilebilsin? (Sözlükte olan kelime 20 bit, olmayan kelime 48 bit, sıkıştırmasız kelime 40 bit.)

Çözüm (derste verildi): Sözlükte bulunma olasılığı P olsun. $N[20P+48(1-P)] = 40N$ eşitliğinden $-28P + 48 = 40$, $28P = 8$, $P \approx 0{,}29$. Kelimelerin %29'undan fazlası sözlükte olmalıdır ($P > 0{,}29$).

Soru 2 (ekrandan, 00:40:00): $P(a_1) = 0{,}8$ ve $P(a_2) = 0{,}2$ olan alfabeden üç sembollü string'ler oluşturuluyor. Olasılık değerleri ve Huffman kodları tablodaki gibidir. Ortalama olasılığı, varyansı ve sembol başına ortalama kod uzunluğunu bulun.

Çözüm (derste verildi): Ortalama olasılık $1/8 = 0{,}125$. Varyans $|0{,}512 - 0{,}125| + 3|0{,}128 - 0{,}125| + 3|0{,}032 - 0{,}125| + |0{,}008 - 0{,}125| = 0{,}792$. Ortalama Huffman kod uzunluğu $1(0{,}512) + 3 \cdot 3(0{,}128) + 3 \cdot 5(0{,}032) + 5(0{,}008) = 2{,}184$ bit/string; sembol başına $2{,}184/3 = 0{,}728$ bit.

Soru 3 (derste sözlü, 01:00:27): Search buffer'ın uzunluğu $S = 32$ ise offset için kaç bit ayrılır?

Çözüm (derste verildi): $32 = 2^5$ olduğundan $\lceil \log_2 32 \rceil = 5$ bit.

Soru 4 (derste sözlü, 01:38:47): "sir sid eastman easily teases sea sick seals" metninin LZ78 kodlamasında 14. satır "y" (0, y) olduktan sonra 15. satırda ne olur?

Çözüm (derste verildi): Sırada boşluk vardır ve sözlükte 4. sıradadır. "␣t" sözlükte yoktur. 15. satıra "␣t" yazılır, token (4, t) olur.

Soru 5 (ekrandan, 02:08:45): "abbabbaabb" girdisini, alfabe {a, b} ve codebook boyu 5 iken LZW ile kodlayın. Codebook sınırlı olmasaydı ne olurdu?

Çözüm (derste verildi): Codebook: 0 a, 1 b, 2 ab, 3 bb, 4 ba. Çıkış 0 1 1 2 4 2 1. Sınırsız codebook'ta 5 abb, 6 baa eklenir ve çıkış 0 1 1 2 4 5 olur. Python ile doğrulandı.

Soru 6 (derste sözlü, 02:08:51): Aynı girdi ("abbabbaabb") LZ78 ile nasıl kodlanır?

Çözüm (derste verilenden düzeltildi, python ile doğrulandı): 1 a (0, a); 2 b (0, b); 3 ba (2, a); 4 bb (2, b); 5 aa (1, a); kalan "bb" 4. sırada olduğundan son token (4, -). Derste tahtaya yazılan çözüm bir karakterin yanlış okunması yüzünden farklıydı (bkz. Bölüm 8.2).

Soru 7 (derste sözlü, 02:11:08): Aynı girdi LZ77 ile kodlansaydı token'lar ne olurdu? (Derste yalnız ilk üç token gösterildi.)

Çözüm (derste verilmedi): Offset sağdan, en yakın sembol 1 olacak biçimde sayılırsa: (0, 0, a), (0, 0, b), (1, 1, a), (3, 3, a), ardından kalan "bb" için arama alanında 1 sembollük eşleşme ve son sembol: (7, 1, b). Python ile hesaplandı; aynı uzunlukta birden fazla eşleşme olduğunda slayttaki gibi en uzaktaki eşleşme seçildi. Son sembolün nasıl ele alınacağı uygulamaya göre değişebilir.

Dersin Toparlaması

Bu derste olasılık tabanlı yöntemlerden sözlük tabanlı yöntemlere geçildi. Önce en basit durum olan statik sözlük görüldü: İndeks ile ham kelimeyi ayırmak için bir bayrak biti gerekir ve kazanç için kelimelerin yaklaşık %29'undan fazlasının sözlükte olması gerekir. String uzunluğu arttıkça sembol başına kod uzunluğunun düştüğü Huffman örneğiyle gösterildi; sözlük yöntemlerinin temel fikri olabildiğince uzun pattern'ları tek seferde kodlamaktır.

Ardından LZ ailesinin üç temel yöntemi anlatıldı. LZ77 önceden kodlanmış veriyi kayan bir pencerede sözlük olarak kullanır ve (offset, uzunluk, sembol) token'ı yazar. LZ78 kayan pencere yerine boş başlayan, trie ile tutulan bir sözlük kullanır ve (sıra, sembol) token'ı yazar. LZW ise sözlüğü alfabeyle başlatıp çıkışa yalnız indeks yazar; LZ'ler içinde en başarılı olanıdır. Son bölümde bu yöntemlerin GIF, TIFF, PNG ve ZIP gibi formatlarda nasıl kullanıldığı ve JPEG'in renk dönüşümü, down-sampling, DCT, nicemleme, zigzag tarama ve entropi kodlamasından oluşan yapısı anlatıldı. Hoca, görüntüye özel bir sıkıştırma yöntemi olmadığını, görüntüye yalnız ön işlem yapılıp metinde görülen yöntemlerin uygulandığını vurguladı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.